Jac and WeBridge: existing SNB benchmarks

Research problem · How do graph-aware SQL lowering and procedure synthesis compare?

Measure two separate transformations: combining graph operations into relational queries, and moving a sequence of database operations into one stored procedure.

Six additional workloads: profile lookup, recent messages, friends, name-filtered traversal, friends’ messages and bounded shortest path.

Each workload has eight configurations, original-parameter correctness gates, injected-delay timing, and an observed SQL / synchronization map.

135 measured latency medians · 27 censored first requests · detailed IC5 appendix

Evidence: results.json; independent verification receipts; per-request outputs and protocol traces

A concrete example: recent messages

IS2: person → ten recent messages → reply roots → authors

Stage	Observed Jac database work	Conventional SQL
Recent messages	One explicit jtopk call; lateral topK	ORDER BY + LIMIT 10
Reply roots	One explicit jroot_many call; promoted root column	Join root message mirror
Author information	Projection + traversal + anchor loads	Join message and root authors
Complete request	24 ordinary SQL cycles	4 commands: one business SELECT + 3 transaction commands

The source already asks for topK and batched roots. The experiment identifies which stages are already database operators and which loads remain separate.

Evidence: unchanged Jac app; native/profiles/operation-map.json; conventional frozen SQL and gates

Solutions under test

The same logical outputs, with explicitly different execution boundaries

Configuration pair	What the source expresses	What is measured
Jac original / final	Existing Jac walker and runtime operators	Change from c12 to final32
Java full / WeBridge full	Native ordinary SQL sequence, including identity, ACL and hydration	Direct requests vs actual generated CALL
Java pruned / WeBridge pruned	Graph calculation with declared Jac overhead removed	Direct requests vs actual generated CALL
Set-oriented Java / WeBridge set-oriented	Conventional joins, projection, topK and bounded path queries	Direct queries vs actual generated CALL

Jac uses PostgreSQL; Java and WeBridge use MySQL 5.7. Within each pair, source behavior, dataset and database engine are held fixed.

Evidence: source snapshots, actual procedure activation, canonical hash gates

Matched data and original inputs

1,475,843 anchors · all Person and Knows records · complete query-specific message closure

Each query retains its original twenty parameter rows. Performance and protocol profiles use original row zero; all six outputs are nonempty.

The fixture contains authored messages, ReplyOf ancestors, authors, locations, endpoints and owner roots needed by all 120 query inputs. Thirteen logical anchor fields are verified after loading both engines.

Native original/final: 240 result gates. Java full/pruned: 240 result gates. Conventional SQL: 120 result gates. Each timing request checks its actual output.

Evidence: expansion/fixture receipts; inputs-all20.json; historical-hashes.json; independent gates

Measurement and attribution

Quiet execution timing is collected separately from instrumented SQL/protocol profiles.

Measurement	Definition
Latency matrix	0 / 2 / 10 ms added delay; fast cells: 10 warmup + 30 measured; slow CALLs: three bounded 15-second attempts per condition
Jac timer	Server request span + commit; HTTP client time retained separately
Java timer	JDBC request span; thread CPU recorded separately
Synchronization cycle	Command sent → entire database response delivered; next command follows completion
Inside CALL	Actual nested table operations, cursor materializations, generated metadata and control events
Compilation	Original recording/replay/synthesis/install/activation stages; failures and compatibility changes retained

Evidence: native/latency; Java timing raw rows; calibration and cleanup receipts

Six new queries: native SQL does not change

Original and final emit the same ordinary SQL sequence for each representative input.

Query	Workload	Original cycles	Final cycles
IS1	Person profile and location	7	7
IS2	Recent messages, reply roots and authors	24	24
IS3	Friends and projected person fields	12	12
IC1	Name-filtered friends within three hops	13	13
IC2	Friends’ recent messages and authors	33	33
IC13	Shortest path with a six-hop cap	14	14

IC13 also executes one periodic schema TTL guard in one +2 ms profile per variant: actual totals reach 15 instead of 14. All other profiles retain the ordinary count.

Evidence: native/profiles/ordinary-sql-equality.json; operation-map.json; 72 measured requests

Generated procedures: actual execution coverage

Every attempted configuration retains stage receipts and frozen source hashes.

Query	Full WeBridge	Pruned WeBridge	Set-oriented WeBridge
IS1	CALL verified	CALL verified	CALL verified
IS2	CALL verified	CALL verified	CALL verified
IS3	CALL verified	CALL verified	CALL verified
IC1	Generated; CALL >15s	Generated; CALL >15s	Generated; CALL >15s
IC2	CALL verified	CALL verified	CALL verified
IC13	CALL verified	CALL verified	CALL verified

IC1 full / IC2 full emit usable SQL and runtime definitions before a recorder trace-hash stack overflow. The original artifacts are installed and tested; IC2 full completes, while IC1 full reaches the request limit.

Evidence: results.json pipeline observations; original compiler logs and compatibility overlays

IS1 — latency at three response delays

Person profile and location · median milliseconds

Configuration	0 ms	+2 ms	+10 ms	Measured sync cycles
Jac original	2.01	18.08	79.84	7
Jac final	2.05	17.57	79.59	7
Java full	2.41	18.81	80.70	7
WeBridge full	2.92	11.74	45.32	4
Java pruned	1.87	12.68	57.55	5
WeBridge pruned	2.01	11.27	50.03	4
Java set-oriented	0.98	10.50	44.65	4
WeBridge set-oriented	1.23	10.64	48.20	4

IS1 — what is combined, what remains?

Person profile and location

Native operation category	Executions
traversal.generic	1
hydration.full_anchor	3

Jac emits three full-anchor loads and one location traversal, alongside three transaction commands. The original/final sequence is identical.

Compare the actual generated CALL against the full and pruned direct sequences; the nested operation map records retained table operations.

Conventional SQL combines the Person/location projection with a LEFT JOIN in one business query. Full object hydration remains separate in Jac.

Evidence: native SQL catalog; actual wire command sequences; executed P_S business operation maps

IS2 — latency at three response delays

Recent messages, reply roots and authors · median milliseconds

Configuration	0 ms	+2 ms	+10 ms	Measured sync cycles
Jac original	6.44	68.23	272.71	24
Jac final	6.41	66.83	270.71	24
Java full	6.22	62.73	270.26	24
WeBridge full	5.26	13.68	48.71	4
Java pruned	2.44	18.64	81.82	7
WeBridge pruned	3.36	11.99	46.22	4
Java set-oriented	1.54	11.47	45.34	4
WeBridge set-oriented	1.89	10.96	44.38	4

IS2 — what is combined, what remains?

Recent messages, reply roots and authors

Native operation category	Executions
reply_root.promoted_column	1
projection.person_fields_acl	1
traversal.generic	1
identity.guest_resolution	2
hydration.full_anchor	15
topk.lateral	1

Jac already uses explicit jtopk and jroot_many builtins: lateral topK and promoted reply roots. Fifteen full-anchor loads, author projection/traversal and guest resolution remain.

The Java port preserves the ordinary native sequence. Its MySQL topK adaptation uses finite per-origin branches because MySQL 5.7 lacks LATERAL.

One conventional query joins the message mirror, reply-root message and both authors, orders by date/ID, and limits to ten. All twenty historical outputs pass.

Evidence: native SQL catalog; actual wire command sequences; executed P_S business operation maps

IS3 — latency at three response delays

Friends and projected person fields · median milliseconds

Configuration	0 ms	+2 ms	+10 ms	Measured sync cycles
Jac original	3.33	33.40	136.97	12
Jac final	3.27	35.42	137.07	12
Java full	4.73	31.77	139.82	12
WeBridge full	5.13	12.90	45.76	4
Java pruned	2.78	14.02	56.58	5
WeBridge pruned	3.05	11.75	45.26	4
Java set-oriented	1.66	10.95	46.46	4
WeBridge set-oriented	2.10	11.19	44.85	4

IS3 — what is combined, what remains?

Friends and projected person fields

Native operation category	Executions
acl.root_permissions	2
type_registry	1
identity.guest_resolution	2
hydration.full_anchor	2
projection.graph_hop_acl	2

Jac already projects friend fields in two directional Knows queries. Root/Person loads, type discovery, root permissions and guest resolution remain separate.

Actual full capture required column aliases and scalar JSON/type metadata compatibility fixes. Each attempt and the selected source snapshot are retained.

Conventional SQL combines both incident-edge directions and Person projection in one UNION ALL query. It returns the same twenty historical results.

Evidence: native SQL catalog; actual wire command sequences; executed P_S business operation maps

IC1 — latency at three response delays

Name-filtered friends within three hops · median milliseconds

Configuration	0 ms	+2 ms	+10 ms	Measured sync cycles
Jac original	37.60	68.81	184.09	13
Jac final	37.44	68.51	189.19	13
Java full	185.08	214.64	337.79	13
WeBridge full	timeout >15s	timeout >15s	timeout >15s	incomplete CALL
Java pruned	184.12	200.35	282.09	8
WeBridge pruned	timeout >15s	timeout >15s	timeout >15s	incomplete CALL
Java set-oriented	269.24	284.88	361.32	7
WeBridge set-oriented	timeout >15s	timeout >15s	timeout >15s	incomplete CALL

Generated SP cells: three fresh first requests per delay; warmup remains incomplete; 15-second request limit.

IC1 — what is combined, what remains?

Name-filtered friends within three hops

Native operation category	Executions
projection.person_fields_acl	1
closure.endpoint_filter	1
closure.frontier_adjacency	3
identity.guest_resolution	2
hydration.full_anchor	3

Jac performs three adjacency-frontier expansions, endpoint filtering and a Person projection. Three anchor loads and two guest lookups remain separate. The final sequence matches the original.

This case measures synthesis over an actual frontier-based graph traversal. Capture and synthesis outcomes are recorded at their executed stages.

The measured conventional reference retains frontier BFS. A finite-depth UNION alternative passed twenty outputs but had a higher diagnostic median, so BFS was selected.

Evidence: native SQL catalog; actual wire command sequences; executed P_S business operation maps

IC2 — latency at three response delays

Friends’ recent messages and authors · median milliseconds

Configuration	0 ms	+2 ms	+10 ms	Measured sync cycles
Jac original	10.53	97.36	379.50	33
Jac final	10.60	100.31	385.70	33
Java full	23.35	103.52	392.59	33
WeBridge full	22.39	31.23	72.78	4
Java pruned	19.90	34.65	100.21	7
WeBridge pruned	19.82	28.04	68.99	4
Java set-oriented	15.53	26.61	74.89	5
WeBridge set-oriented	15.35	24.40	68.81	4

IC2 — what is combined, what remains?

Friends’ recent messages and authors

Native operation category	Executions
type_registry	1
traversal.generic	3
hydration.full_anchor	25
topk.lateral	1

Jac already requests topK explicitly. Twenty-five full-anchor loads, two Knows traversals, a HasCreator traversal and a type probe remain separate.

Full Java retains the observed object-loading sequence; the declared pruned form removes Jac-specific loading while preserving the query output.

Conventional SQL first selects friends, then unions bounded per-author/type top-20 candidates and applies global top-20. Both candidate and selected outputs were measured against all twenty historical hashes.

Evidence: native SQL catalog; actual wire command sequences; executed P_S business operation maps

IC13 — latency at three response delays

Shortest path with a six-hop cap · median milliseconds

Configuration	0 ms	+2 ms	+10 ms	Measured sync cycles
Jac original	35.14	69.98	216.23	14–15
Jac final	34.99	70.25	210.08	14–15
Java full	168.09	200.80	326.09	14
WeBridge full	2247.41	2232.18	2314.68	4
Java pruned	166.79	183.27	263.40	8
WeBridge pruned	2256.63	2243.12	2291.72	4
Java set-oriented	1.53	11.03	44.45	4
WeBridge set-oriented	1.90	11.24	44.48	4

IC13 — what is combined, what remains?

Shortest path with a six-hop cap

Native operation category	Executions
projection.person_fields_acl	1
identity.guest_resolution	2
closure.endpoint_filter	1
closure.frontier_adjacency	3
schema.ttl_guard	0–1
hydration.full_anchor	4

Actual execution uses indexed adjacency-frontier BFS. Three frontiers reach the selected endpoint; a periodic schema guard can add one cycle.

Full/pruned generated BFS procedures contain about 4.85 million row-index checks at the captured cursor sizes. Their actual quiet latencies and observed table operations are reported separately.

The conventional alternative uses ordered CASE/EXISTS for lengths zero through six, matching the benchmark cap. All twenty outputs pass; its diagnostic median was lower than the BFS candidate.

Evidence: native SQL catalog; actual wire command sequences; executed P_S business operation maps

IS1 — executed WeBridge operations

Actual nested business operations, return work and client/server traffic

Actual operation category	Full	Pruned	Set-oriented
anchor.projection	0	1	0
graph.IsLocatedIn	1	1	1
hydration.full_anchor	3	0	0
Return / metadata SELECTs	3	1	1
Control instrumentation events	111	40	8

Variant	Direct cycles	CALL cycles	CALL frames	CALL result sets	CALL bytes
full	7	4	76	5	5098
pruned	5	4	33	3	1581
set	4	4	14	2	961

Evidence: mysql-profile/summary.json; source-associated cursor opens; complete observed CALL ranges

IS2 — executed WeBridge operations

Actual nested business operations, return work and client/server traffic

Actual operation category	Full	Pruned	Set-oriented
anchor.projection	1	1	0
graph.HasCreator	1	1	0
graph.ReplyOf	1	1	1
hydration.full_anchor	15	0	0
identity.lookup	2	0	0
message.topk	1	1	0
Return / metadata SELECTs	8	3	1
Control instrumentation events	606	286	8

Variant	Direct cycles	CALL cycles	CALL frames	CALL result sets	CALL bytes
full	24	4	333	22	29801
pruned	7	4	76	5	11416
set	4	4	27	2	21695

Evidence: mysql-profile/summary.json; source-associated cursor opens; complete observed CALL ranges

IS3 — executed WeBridge operations

Actual nested business operations, return work and client/server traffic

Actual operation category	Full	Pruned	Set-oriented
acl.root_permissions	2	0	0
graph.Knows	2	2	1
hydration.full_anchor	2	0	0
identity.lookup	2	0	0
type_registry	1	0	0
Return / metadata SELECTs	2	1	1
Control instrumentation events	162	64	10

Variant	Direct cycles	CALL cycles	CALL frames	CALL result sets	CALL bytes
full	12	4	127	10	12995
pruned	5	4	55	3	5091
set	4	4	25	2	1064

Evidence: mysql-profile/summary.json; source-associated cursor opens; complete observed CALL ranges

IC1 — executed WeBridge operations

Actual nested business operations, return work and client/server traffic

Actual operation category	Full	Pruned	Set-oriented
Return / metadata SELECTs	—	—	—
Control instrumentation events	—	—	—

Variant	Direct cycles	CALL cycles	CALL frames	CALL result sets	CALL bytes
full	13	Generated; CALL >15s	—	—	—
pruned	8	Generated; CALL >15s	—	—	—
set	7	Generated; CALL >15s	—	—	—

Evidence: mysql-profile/summary.json; source-associated cursor opens; complete observed CALL ranges

IC2 — executed WeBridge operations

Actual nested business operations, return work and client/server traffic

Actual operation category	Full	Pruned	Set-oriented
closure.frontier_adjacency	0	0	1
graph.HasCreator	1	1	0
graph.Knows	2	2	0
hydration.full_anchor	25	0	0
message.topk	1	1	1
type_registry	1	0	0
Return / metadata SELECTs	25	3	2
Control instrumentation events	not collected	994	244

Variant	Direct cycles	CALL cycles	CALL frames	CALL result sets	CALL bytes
full	33	4	750	31	76121
pruned	7	4	235	5	36137
set	5	4	85	3	25037

Evidence: mysql-profile/summary.json; source-associated cursor opens; complete observed CALL ranges

IC13 — executed WeBridge operations

Actual nested business operations, return work and client/server traffic

Actual operation category	Full	Pruned	Set-oriented
anchor.projection	1	1	0
closure.bounded_case_exists	0	0	1
closure.frontier_adjacency	3	3	0
hydration.full_anchor	4	0	0
identity.lookup	2	0	0
person.projection	1	1	0
Return / metadata SELECTs	4	3	1
Control instrumentation events	not collected	not collected	34

Variant	Direct cycles	CALL cycles	CALL frames	CALL result sets	CALL bytes
full	14	4	13561	12	762869
pruned	8	4	13485	6	752040
set	4	4	26	2	1198

Evidence: mysql-profile/summary.json; source-associated cursor opens; complete observed CALL ranges

Where does CALL time go?

Direct-child instrumentation timers; parent CALL duration includes descendants

Query	Variant	CALL ms	Business ms	Control ms	Return ms
IC13	conventional	0.861	0.749	0.023	0.021
IC13	full	2290.671	160.196	not collected	3.949
IC13	pruned	2265.886	158.538	not collected	4.143
IC2	conventional	13.713	13.296	0.130	0.055
IC2	full	19.176	16.175	not collected	0.592
IC2	pruned	17.410	16.121	0.540	0.140
IS1	conventional	0.264	0.191	0.008	0.019
IS1	full	1.136	0.809	0.102	0.062
IS1	pruned	0.821	0.689	0.031	0.019
IS2	conventional	0.439	0.369	0.008	0.013
IS2	full	2.423	1.347	0.429	0.166
IS2	pruned	1.155	0.694	0.190	0.068
IS3	conventional	0.863	0.795	0.010	0.016
IS3	full	2.025	1.639	0.135	0.050
IS3	pruned	1.522	1.325	0.047	0.025

These instrumented timings locate work inside successful CALLs. The quiet latency matrix uses separate runs with stored-program instrumentation disabled.

Evidence: mysql-profile/summary.json; three complete postwarm CALL ranges per displayed row

IC1: fewer commands can still perform more SQL work

Same MySQL fixture and input · quiet request medians beside separate instrumented statement medians

Java variant	Quiet 0ms request	Profile frontier ms	Profile final query ms	Final query rows examined	Final query UUIDs
full	185.08	176.25	0.24	18	18
pruned	184.12	177.55	0.24	18	18
conventional	269.24	177.49	86.64	9932	9036

Full/pruned preselect twenty name-matching candidate IDs, then project eighteen surviving people. The conventional query filters after BFS using 9,036 visited UUIDs and examines all 9,932 Person rows.

All three run the same frontier inputs: 1 / 10 / 2,740 UUIDs; frontier statements examine 175,262 rows and return 11,769 rows in each request.

Evidence: ic1-same-engine-comparison.json; exact wire SQL hashes, UUID cardinalities and P_S rows/timers

IC13: measured time beyond table statements

Actual completed CALLs with business-statement instrumentation; control events are unobserved

Variant	Quiet 0ms request	Profile CALL ms	Profile child statements ms	Remaining CALL interval ms
full	2247.41	2290.671	164.126	2126.545
pruned	2256.63	2265.886	162.593	2103.747
conventional	1.90	0.861	0.793	0.067

About 2.1 seconds remain inside the full/pruned procedures beyond the instrumented child statements. This interval includes uninstrumented cursor/control and other procedure runtime work.

The source has about 4.85 million row-index checks at recorded cursor sizes. The conventional scalar CASE/EXISTS procedure has a much smaller remaining interval.

Evidence: ic13-cost-breakdown.json; business-only settings, observed P_S event ranges and cursor-source association

Intermediate results shape generated control work

Source-derived row-index checks at recorded cursor cardinalities, beside measured quiet latency

Query	Full static checks	Pruned static checks	Full quiet ms	Pruned quiet ms
IS1	4	0	2.92	2.01
IS2	134	116	5.26	3.36
IS3	2	0	5.13	3.05
IC1	88,736,452	88,736,450	timeout >15s	timeout >15s
IC2	972	510	22.39	19.82
IC13	4,851,248	4,851,246	2247.41	2256.63

IC13 captures large BFS intermediate results in row-specific variables. The conventional bounded CASE/EXISTS query performs path tests inside one SQL statement and returns a scalar.

Evidence: java/cursor-dispatch-analysis.json → hashed generated SQL; results.json → quiet request rows

IC1: why one CALL can still be expensive

Measured generated-code behavior · the pruned procedure retains frontier BFS results

Cursor loop	Independent IF branches	Loop visits through exit	Equality checks
1	10	11	110
2	2,750	2,751	7,565,250
3	9,009	9,010	81,171,090

Total generated equality checks: 88,736,450. Each fetched row scans all row-specific IF branches before the loop advances.

One instrumented CALL completed correctly: 129.396 seconds inside the server. A separate quiet run with stored-program instrumentation disabled exceeded the 15-second limit and was terminated.

Evidence: java/runaway-ic1/loop-structure.json; completed report-0.json; server P_S counters

IC1: the generated dispatch mechanism

A bounded cursor loop with a separate destination variable for every captured row

```
loop_3: LOOP
  IF cursor_cnt_3=0 THEN FETCH cursor_3 INTO RS_q4_r1_col1; END IF;
  IF cursor_cnt_3=1 THEN FETCH cursor_3 INTO RS_q4_r2_col1; END IF;
  ...
  IF cursor_cnt_3=9008 THEN FETCH cursor_3 INTO RS_q4_r9009_col1; END IF;
  IF cursor_cnt_3>=9009 THEN CLOSE cursor_3; LEAVE loop_3; END IF;
  ...
  SET cursor_cnt_3 = cursor_cnt_3 + 1;
END LOOP;
```

The three source adjacency queries remain three database queries inside CALL. The large intermediate frontiers also produce row-specific procedural control work.

Quiet slow-arm cells preserve three actual bounded attempts per delay condition. Completed latency samples and censored attempts have separate labels and raw receipts.

Evidence: actual generated procedure cited in loop-structure.json; branch counts 10 / 2,750 / 9,009

Compare with IC5: a different observed optimization

Existing IC5 evidence uses its own 460,984-anchor matched fixture.

Observation	IC5	Six additional queries
Jac ordinary SQL count	38 → 14	Unchanged in each original/final pair
Actual Jac combination	10 membership queries → 1; 16 post/creator queries → 1 aggregate	Existing topK, projections, promoted roots and adjacency expansion
WeBridge transformation	35 full / 28 pruned business operations execute inside one CALL	Per-query executed operations and capture-stage receipts
Conventional baseline	One aggregate SQL + 3 transaction commands	Query-specific joins, topK and bounded path SQL

The detailed IC5 experiment follows as an appendix, with its complete eight-configuration matrix and nested operation counts.

Evidence: roundtrip-comparison/eight-arm-verification.json and webridge-operation-map.json

Evidence and reproduction

Raw outputs, timing rows, SQL catalogs, procedure sources and measured failure stages

Inputs	Frozen twenty-row inputs and historical result hashes; fixture export/load SHA receipts.
Correctness	Actual result objects rehashed against the frozen oracle; activation and driver/server counts checked.
Performance	All measured requests and calibration controls; original timer boundaries and warmup labels.
SQL and protocol	Complete-response sequencing, SQL categories, nested executed work, generated procedure hashes.
Compilation	Captured attempts, precise failed stages, isolated compatibility diffs and selected immutable cases.

Evidence: archive-manifest.json; benchmark-comparison/verify_results.py; results.json

Jac and WeBridge: measured SQL fusion and database round trips

Two complete experiments · eight configurations · fixed nonempty IC5 input

Experiment 1: request latency at 0, 2 and 10 ms injected response delay.

Experiment 2: the exact SQL operations and serialized command/response cycles for every configuration.

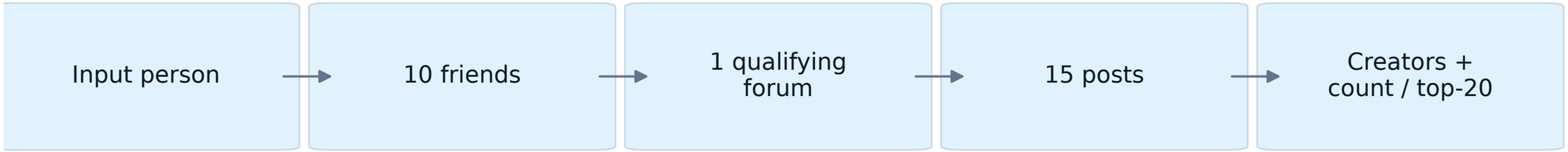
Jac emits fewer business queries through batching and relational aggregation. WeBridge full/pruned puts the original business-query sequence inside one CALL. Set-oriented Java already expresses the graph calculation as one SQL query.

960 main timing requests + 240 reverse-order requests; 30 measured protocol profiles.

Evidence: eight-arm-verification.json; webridge-operation-map.json; Jac roundtrip-profile/results/operation-map.json

One input and matched logical graph data

460,984 anchors · 300,901 edges · exact-result checks in every timing request



personId = 8796093030404; minDate = 1347527525888

Expected: forum 2061584375281, “Album 7 of Jie Xie”, postCount 0.

Jac uses PostgreSQL 18.6 and runtime mirror/adjacency structures. Java uses MySQL 5.7.44 over generic anchors. Cross-system timing compares these complete implementations.

Evidence: alignment/small-diagnostic/input.json; jac-comparison/fixture/load-receipt.json; correctness gates

Experiment 1 — complete latency matrix

Median execution time (ms) · 10 warmup + 30 measured requests per cell

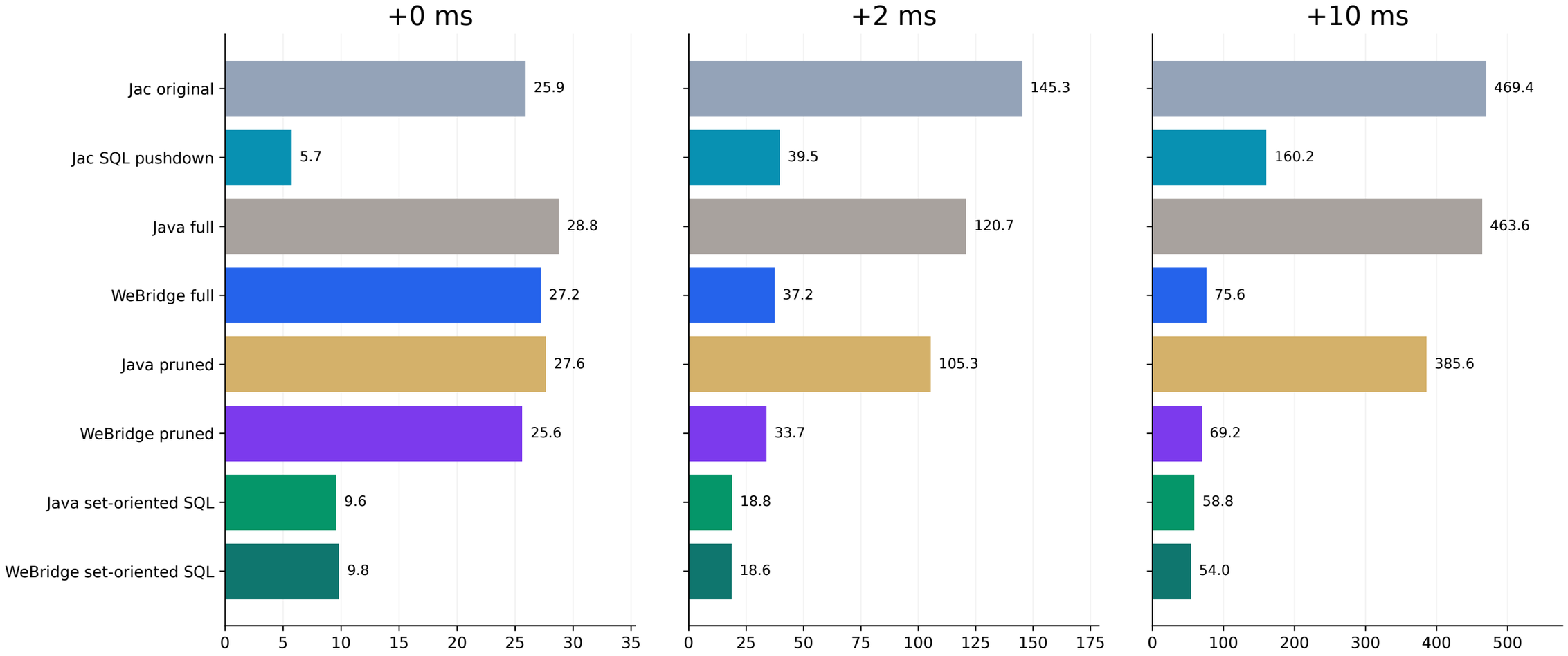
Configuration	0 ms	+2 ms	+10 ms	SQL / sync cycles
Jac original	25.89	145.26	469.42	38
Jac SQL pushdown	5.73	39.53	160.23	14
Java full	28.75	120.74	463.64	38
WeBridge full	27.20	37.25	75.63	4
Java pruned	27.64	105.27	385.60	31
WeBridge pruned	25.62	33.70	69.23	4
Java set-oriented SQL	9.56	18.79	58.82	4
WeBridge set-oriented SQL	9.81	18.61	53.96	4

All settings use the same delay-proxy path. Jac: server-total + commit timer. Java: JDBC request timer. Both exclude response JSON serialization.

Evidence: eight-arm-verification.json and raw per-request rows; calibrated SELECT 1 receipts

Experiment 1 — latency under added delay

Each panel has its own millisecond scale; values come from the complete matrix.



Evidence: eight-arm-verification.json · raw timing is separate from instrumented profiles

Set-oriented Java through actual WeBridge

Original recording → original Zero JVM → synthesis → generated procedure → execution

Delay	Direct, first	CALL, second	Direct, second	CALL, first
+0 ms	9.56	9.81	9.57	10.19
+2 ms	18.79	18.61	18.51	18.83
+10 ms	58.82	53.96	55.40	57.73

Both paths make four command/response cycles. At 10 ms the ordering reverses when phase order is reversed. These measurements show no consistent WeBridge speedup for the already set-oriented query.

All 20 original parameterized stored-procedure result gates pass. Both timing orders pass all 480 requests, with one CALL only in the optimized arm.

Evidence: alignment/idiomatic/WEBRIDGE.md; webridge-independent-verification.json; generated SQL SHA256

Experiment 2 — complete measured synchronization matrix

“Round trip” here means a serialized request followed by a complete database response.

Configuration	Sync cycles	Business / runtime reads	Outer txn commands	Replicates
Jac original	38	35 separate reads	3	6
Jac SQL pushdown	14	11 separate reads	3	6
Java full	38	35 separate reads	3	3
WeBridge full	4	35 inside one CALL	3	3
Java pruned	31	28 separate reads	3	3
WeBridge pruned	4	28 inside one CALL	3	3
Java set-oriented SQL	4	1 aggregate SELECT	3	3
WeBridge set-oriented SQL	4	1 aggregate inside CALL	3	3

Evidence: MySQL protocol-analysis.json (three postwarm requests per arm); PG flight-summary.json (six per variant)

Jac: which operations are combined, and which remain?

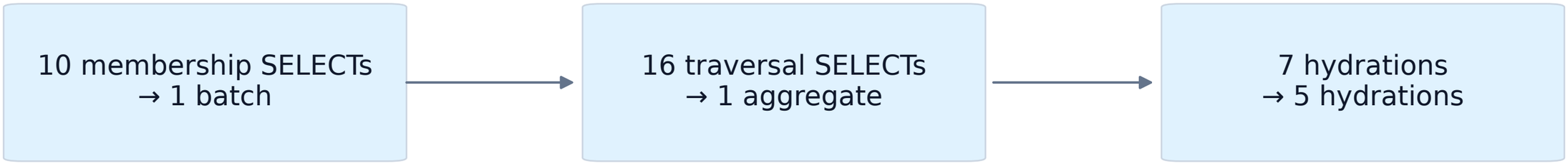
Exact warm PostgreSQL statement sequence; the original and final variants use the same walker.

Operation	Original	Final	Measured emitted form
Transaction start / setting / commit	3	3	Separate
Anchor hydration	7	5	Root, person, two friend batches, forum remain
Knows traversals	2	2	Outgoing and incoming remain separate
HasMember traversals	10	1	One batch over ten friends
Standalone type-ancestry probes	0	2	HasMember and Forum probes remain
ContainerOf + HasCreator	1 + 15	0	Combined into aggregate query below
Count / ordering / LIMIT SQL	0	1	Member-driven relational aggregate
Total SQL / synchronization cycles	38	14	All observed in each profile request

Evidence: Jac roundtrip-profile/results/operation-map.json and sql-catalog.json; protocol + backend SQL logs

Jac: account for every removed and retained cycle

The observed reduction is exactly 38 → 14.



38 − 9 − 15 − 2 + 2 type probes = 14

The two Knows traversals remain separate. Five anchor-loading queries and both type probes remain separate. The single forum is hydrated before the fused count query in this measured sequence.

WeBridge full/pruned moves their corresponding original queries into CALL; its executed category counts are on the next page.

Evidence: operation-map.json; named prepared statement catalog; bound frontier counts and result command tags

WeBridge: exact business operations executed inside CALL

Direct table SELECTs and executed cursor materializations are matched to generated procedure SQL.

Operation	WeBridge full	WeBridge pruned	WeBridge set-oriented
Hydration SELECTs	7	0	0
Knows traversals	2	2	part of aggregate
HasMember traversals	10	10	part of aggregate
ContainerOf traversals	1	1	part of aggregate
HasCreator traversals	15	15	part of aggregate
Set-oriented aggregate	0	0	1
Total business table operations	35	28	1
Outer transaction commands	3	3	3

Evidence: webridge-operation-map.json; actual P_S child events, cursor IDs and generated declarations

What each transformation achieves in this request

Compare the logical query boundary separately from the client/server boundary.

Comparison	Jac relational combination	Queries / boundaries still separate
Jac vs WeBridge full	Membership 10→1; post/creator 16→1; hydration 7→5	Jac retains 14 client cycles; WeBridge full uses 4
Jac vs WeBridge pruned	Jac batches membership and fuses count/order/LIMIT; pruned CALL retains 28 traversal operations	Pruned removes all hydration; Jac retains 5 hydration + 2 type probes
Jac vs set-oriented Java / CALL	Both have database-side aggregation and top-20	Set-oriented SQL also includes friend and membership joins in its single query; Jac emits them separately

Within the measured capture region, WeBridge full/pruned captures every original business SELECT. START, SET and COMMIT remain outside. The set-oriented variant captures its one aggregate SELECT.

Evidence: SQL operation maps and actual command sequences for all eight configurations

Protocol frames and return traffic are different from round trips

Actual MySQL frame decoding honors negotiated CLIENT_DEPRECATE_EOF.

MySQL configuration	Sync cycles	Response frames	Result sets	Response bytes
Java full	38	276	35	29,286
WeBridge full	4	970	36	68,300
Java pruned	31	176	28	6,798
WeBridge pruned	4	608	29	30,547
Java set-oriented SQL	4	9	1	220
WeBridge set-oriented SQL	4	17	2	495

Every next command follows the prior complete response in the trace. One CALL can return many protocol frames and result sets. For PostgreSQL, each stable execution is one Bind/Describe/Execute/Sync → ReadyForQuery cycle; the next Bind follows Ready delivery.

The PostgreSQL baseline uses 39 server socket chunks for 38 cycles; final uses 14 chunks for 14 cycles. Socket chunk count is recorded separately.

Evidence: mysql-results/protocol-analysis.json; mysql-idiomatic-results/protocol-analysis.json; parser regression checks

Inside the stored procedures: original work and generated work

Directly nested Performance Schema events for a complete observed CALL.

Observed event category	Full	Pruned	Set-oriented
Business table SELECTs	25	24	1
Business cursor materializations	10	4	0
Result / parameter metadata SELECTs	11	5	1
Variable / control instrumentation events	2,447	1,510	12

Business totals are 25+10=35 and 24+4=28. Cursor opens materialize original business queries.

Result/parameter SELECTs and variable/control events are generated procedure work.

Control-event totals are stored-program instrumentation counts. The set-oriented procedure executes its original aggregate plus one metadata SELECT and still requires four outer command/response cycles.

Evidence: webridge-operation-map.json · parent CALL timers include children; counts here use direct children only

Evidence and reproduction

Both experiments have a complete eight-configuration result matrix.

Latency	eight-arm-verification.json: 24 main cells, 960 checked requests; 240 additional reverse-order requests.
SQL / protocol	PG operation-map + flight-summary: 12 measured requests; MySQL protocol analyses: 18 measured requests.
Actual WeBridge	Generated procedure SHA256, original capture/replay/synthesis/install receipts, all20 parameter gates.
Independent checks	verify_eight_arms.py; audit_operation_map.py; test_mysql_parser.py; Jac roundtrip-profile/analyze.py.
Raw evidence	Per-request timings, payload-free protocol metadata, complete SQL catalog, P_S events and cleanup receipts.